Descripciones de subtareas - HU 1 a 5 (Sprint 2)

Texto para copiar en el campo Descripción de cada subtarea clonada. El título de la subtarea (Dado que…) no cambia.

HU 1 - (1) Yo como AUTOR necesito crear una nueva pregunta para poder incorporarla al banco de preguntas

SSCRUM-34 Dado que creo una pregunta con los datos requeridos, el sistema la guarda exitosamente en estado "Borrador" y confirma la acción.

1. Diseñar la estructura de persistencia propia del microservicio de Preguntas para almacenar una pregunta con contexto, pregunta directa, cuatro opciones, respuesta correcta, justificación, bibliografía, competencia, tema, subtema, nivel de dificultad y estado, sin compartir tablas con otros servicios.
2. Crear las clases de dominio Pregunta y Opción con sus atributos y relaciones, aplicando el patrón Builder para construir la pregunta.
3. Implementar el endpoint POST /preguntas en el microservicio y documentarlo en Swagger.
4. Implementar en la capa de servicios la lógica que invoca al microservicio de Validación estructural y registra la pregunta asignándole el estado "Borrador".
5. Implementar en el frontend (separado del backend) el formulario de creación con los campos requeridos, consumiendo la API REST a través del API Gateway y mostrando la confirmación.
6. Crear pruebas unitarias automatizadas para las clases de dominio y servicios involucrados en la creación y almacenamiento de la pregunta.

SSCRUM-35 Dado que intento guardar una pregunta con campos requeridos incompletos, el sistema bloquea el guardado...

1. Implementar en el microservicio de Validación estructural las reglas (ReglaValidación) que verifican que los campos requeridos estén completos.
2. Devolver una respuesta 400 con la lista de errores (ErrorValidación) y documentarla en Swagger; el servicio de Preguntas no guarda nada si la validación falla.
3. Mostrar en el frontend los errores devueltos por la API y resaltar los campos incompletos.
4. Crear pruebas unitarias automatizadas de las reglas de validación y del flujo de rechazo al guardar.

SSCRUM-36 Dado que intento guardar una pregunta sin completar los campos obligatorios, el sistema impide la acción y...

1. Definir en el servicio de Validación la regla de campos obligatorios (contexto, pregunta directa, justificación, opciones) y registrar el ResultadoValidación de cada intento.
2. Garantizar en el servicio de Preguntas que una pregunta que no pasa la validación no se persista ni cambie de estado.
3. Mostrar en el frontend un mensaje de error general y resaltar cada campo obligatorio faltante.
4. Crear pruebas unitarias automatizadas de la regla y del servicio de registro.

SSCRUM-37 Dado que intento guardar una pregunta sin seleccionar una respuesta correcta, el sistema bloquea el guardado...

1. Implementar en el servicio de Validación la regla que exige exactamente una respuesta correcta entre las cuatro opciones.
2. Devolver un error 400 específico cuando no hay respuesta correcta (o hay más de una) y documentarlo en Swagger.
3. Mostrar en el frontend el mensaje de error y resaltar las opciones.
4. Crear pruebas unitarias automatizadas de la regla con cero, una y varias respuestas correctas.

SSCRUM-38 Dado que guardo sin seleccionar la dificultad, el sistema bloquea la acción, alerta el error y resalta las opciones.

1. Implementar en el servicio de Validación la regla que exige un nivel de dificultad de los valores permitidos.
2. Devolver un error 400 identificando el campo dificultad y documentarlo en Swagger.
3. Mostrar en el frontend la alerta y resaltar las opciones de dificultad.
4. Crear pruebas unitarias automatizadas de la regla con valor vacío, válido e inválido.

SSCRUM-39 Dado que estoy creando una pregunta y el campo de bibliografía de la respuesta está vacío...

1. Implementar en el servicio de Validación la regla que exige bibliografía no vacía.
2. Devolver un error 400 identificando el campo bibliografía y documentarlo en Swagger.
3. Mostrar en el frontend el mensaje de error junto al campo.
4. Crear pruebas unitarias automatizadas de la regla con bibliografía vacía, solo espacios y válida.

SSCRUM-40 Dado que estoy creando una pregunta y no he seleccionado competencia, tema o subtema...

1. Implementar en el servicio de Validación la regla que exige competencia, tema y subtema, y que existan en el catálogo del servicio de Preguntas (consulta por API REST, sin acceder a su BD).
2. Devolver un error 400 indicando cuál de los tres campos falta y documentarlo en Swagger.
3. Cargar en el frontend los catálogos desde la API y bloquear el envío si falta alguno.
4. Crear pruebas unitarias automatizadas de la regla, con el catálogo simulado.

HU 2 - Yo como AUTOR necesito cambiar el estado de mis preguntas de "Borrador" a "Pendiente de revisión"

SSCRUM-42 Dado que envío una pregunta en borrador a revisión, el sistema cambia su estado a "Pendiente de revisión", actualiza su color y confirma el éxito.

1. Crear el microservicio de Ciclo de vida con los estados válidos (EstadoPregunta) y el registro de cada cambio (PreguntaEstado).
2. Aplicar el patrón State para modelar los estados de la pregunta y sus transiciones permitidas.
3. Implementar el endpoint PUT /preguntas/{id}/enviar-revision y documentarlo en Swagger.
4. Publicar en el broker el evento PreguntaEnviadaARevision al confirmar el cambio.
5. Implementar en el frontend el botón de envío, la actualización del color del estado y el mensaje de éxito.
6. Crear pruebas unitarias automatizadas de la transición Borrador a Pendiente de revisión y de la publicación del evento (broker simulado).

SSCRUM-43 Dado que intento enviar a revisión una pregunta que no está en estado "Borrador", el sistema bloquea la acción y muestra un mensaje de error.

1. Implementar en los estados (patrón State) el rechazo de la transición cuando el estado actual no es Borrador.
2. Devolver un error 409 con mensaje claro y documentarlo en Swagger.
3. Mostrar el mensaje de error en el frontend y deshabilitar el botón según el estado.
4. Crear pruebas unitarias automatizadas de las transiciones no permitidas desde cada estado.

SSCRUM-44 Dado que consulto el listado de mis preguntas, el sistema diferencia visualmente con un color distintivo aquellas que están en estado...

1. Definir en el servicio de Ciclo de vida un color por cada estado y exponerlo en la respuesta de la API.
2. Hacer que el servicio de listado incluya el estado y su color en cada pregunta.
3. Mostrar en el frontend cada estado como etiqueta con su color.
4. Crear pruebas unitarias automatizadas de la asignación de color por estado.

SSCRUM-45 Dado que intento enviar a revisión una pregunta que no creé, el sistema bloquea la acción y muestra un mensaje de error por falta de permisos.

1. Validar en el servicio que el usuario del token sea el autor de la pregunta antes de cambiar el estado.
2. Devolver un error 403 con mensaje claro y documentarlo en Swagger.
3. Hacer que el API Gateway verifique el token y el rol antes de enrutar la petición.
4. Mostrar el mensaje de falta de permisos en el frontend.
5. Crear pruebas unitarias automatizadas de la validación de autoría.

SSCRUM-46 Dado que intento editar una pregunta en estado "Pendiente de revisión", el sistema bloquea la edición y muestra un mensaje...

1. Implementar en el servicio de Preguntas la regla de que solo se editan preguntas en estado Borrador, consultando el estado al servicio de Ciclo de vida por API.
2. Devolver un error 409 con mensaje claro y documentarlo en Swagger.
3. Ocultar o deshabilitar la acción de editar en el frontend según el estado y mostrar el mensaje.
4. Crear pruebas unitarias automatizadas de la regla de edición por estado.

SSCRUM-47 Dado que cancelo la confirmación de envío a revisión, el sistema no realiza cambios y mantiene la pregunta en estado "Borrador".

1. Implementar en el frontend el cuadro de confirmación; al cancelar no se llama al endpoint.
2. Verificar que el estado de la pregunta no cambia y que no se publica ningún evento.
3. Crear pruebas automatizadas del flujo de cancelación (sin llamada a la API ni evento).

HU 3 - Yo como AUTOR necesito listar las preguntas que he creado para poder verlas y editarlas

Dado que accedo al listado de mis preguntas, el sistema muestra exclusivamente las creadas por mí, detallando la información clave y el estado de cada una.

1. Implementar el endpoint GET /preguntas que obtenga el autor desde el token y devuelva solo sus preguntas.
2. Aplicar el patrón Facade para unificar en una sola respuesta los datos de Preguntas y el estado del servicio de Ciclo de vida.
3. Documentar el endpoint en Swagger con sus parámetros y respuestas.
4. Implementar en el frontend la tabla del listado con la información clave y el estado de cada pregunta, consumiendo la API a través del gateway.
5. Crear pruebas unitarias automatizadas de la consulta por autor y de la fachada.

Dado que consulto un listado de preguntas que supera el límite por página, el sistema pagina los resultados y permite navegar sin perder los filtros aplicados.

1. Implementar la paginación en el servicio con parámetros de página y tamaño, devolviendo el total de resultados.
2. Mantener los filtros aplicados al cambiar de página.
3. Implementar en el frontend los controles de paginación conservando el estado de los filtros.
4. Crear pruebas unitarias automatizadas de la paginación (primera, intermedia, última página y página vacía).

Dado que filtro el listado de mis preguntas por uno o varios estados, el sistema actualiza los resultados mostrando únicamente las que coinciden.

1. Implementar en el servicio el filtro por uno o varios estados, obteniendo la correspondencia con el servicio de Ciclo de vida.
2. Documentar el parámetro de filtro en Swagger.
3. Implementar en el frontend el selector múltiple de estados y la actualización de resultados.
4. Crear pruebas unitarias automatizadas del filtro por un estado y por varios.

Dado que filtro el listado de mis preguntas por nivel de dificultad, el sistema actualiza los resultados mostrando únicamente las que coinciden.

1. Implementar en el servicio el filtro por nivel de dificultad y combinarlo con los demás filtros.
2. Documentar el parámetro de filtro en Swagger.
3. Implementar en el frontend el selector de dificultad.
4. Crear pruebas unitarias automatizadas del filtro por dificultad y combinado con estado.

Dado que aplico una búsqueda o filtros en mi listado sin obtener coincidencias, el sistema muestra un mensaje indicando que no se encontraron preguntas.

1. Hacer que el servicio devuelva una lista vacía con código 200 (no un error) cuando no hay coincidencias.
2. Mostrar en el frontend el mensaje «No se encontraron preguntas» cuando la lista llega vacía.
3. Crear pruebas unitarias automatizadas del caso sin coincidencias.

Dado que visualizo una pregunta creada por mí, el sistema muestra su información completa y las acciones disponibles según su estado.

1. Implementar el endpoint GET /preguntas/{id} que devuelva la pregunta completa y valide que pertenece al autor.
2. Determinar las acciones disponibles según el estado (editar solo en Borrador, enviar a revisión solo en Borrador).
3. Implementar en el frontend la vista de detalle con las acciones habilitadas según el estado.
4. Crear pruebas unitarias automatizadas de las acciones disponibles por estado.

HU 4 - Yo como ADMINISTRADOR necesito asignar revisores a las preguntas en estado "Pendiente de revisión"

SSCRUM-56 Dado que asigno revisores a una pregunta en revisión como administrador, el sistema guarda la asignación y confirma el cambio.

1. Diseñar la estructura de persistencia propia del microservicio de Revisión para la relación entre preguntas y revisores asignados (Revisión).
2. Crear las clases de dominio necesarias para representar la asignación de entre uno y tres revisores a una pregunta.
3. Implementar el endpoint POST /revisiones/asignar y documentarlo en Swagger.
4. Implementar en la capa de servicios la lógica para validar (entre 1 y 3 revisores) y registrar la asignación.
5. Diseñar e implementar en el frontend la vista del administrador para seleccionar una pregunta y de uno a tres revisores.
6. Crear pruebas unitarias automatizadas para verificar que una pregunta en estado "Pendiente de revisión" pueda recibir correctamente de uno a tres revisores.

Dado que accedo al listado de preguntas para asignar revisor, el sistema solo muestra las preguntas en estado "Pendiente de revisión"...

1. Implementar el endpoint de consulta de preguntas pendientes, que filtre por estado consultando a los servicios de Preguntas y Ciclo de vida por API.
2. Hacer que el API Gateway permita este endpoint solo al rol Administrador.
3. Mostrar en el frontend únicamente las preguntas pendientes en el listado de asignación.
4. Crear pruebas unitarias automatizadas del filtro por estado.

Dado que asigno sin seleccionar un revisor, el sistema bloquea la acción, alerta la falta de selección y resalta el campo.

1. Implementar en el servicio la regla que rechaza asignaciones con cero revisores (y con más de tres), devolviendo un error 400.
2. Documentar los errores en Swagger.
3. Mostrar en el frontend la alerta y resaltar el campo de revisores.
4. Crear pruebas unitarias automatizadas con 0, 1, 3 y 4 revisores.

Dado que accedo al selector de revisores para asignar una pregunta, el sistema solo muestra usuarios con permisos de revisor...

1. Implementar en el microservicio de Usuarios el endpoint que lista usuarios con el rol Revisor.
2. Consumir ese endpoint desde el servicio de Revisión por API REST, sin acceder a la BD de Usuarios.
3. Excluir al autor de la pregunta de la lista de candidatos.
4. Mostrar en el frontend solo a los revisores válidos en el selector.
5. Crear pruebas unitarias automatizadas del filtrado por rol y de la exclusión del autor.

Dado que se confirma la asignación de un revisor a una pregunta, el sistema envía la notificación a cada revisor asignado...

1. Publicar en el broker el evento RevisoresAsignados al confirmar la asignación.
2. Implementar en el microservicio de Notificaciones el consumidor del evento, que envía un email a cada revisor asignado.
3. Aplicar el patrón Factory Method para crear el tipo de notificación y el patrón Adapter para encapsular el proveedor de email.
4. Registrar los fallos de envío y reintentar sin afectar la asignación ya guardada.
5. Crear pruebas unitarias automatizadas de la fábrica de notificaciones y del adaptador (proveedor simulado).

HU 5 - (5) Yo como REVISOR necesito listar las preguntas que me han sido asignadas para poder leerlas, dejar mi evaluación y cambiar su estado a Aprobada/Rechazada

SSCRUM-62 Dado que tengo preguntas asignadas por el administrador, cuando abro mi listado, el sistema me muestra únicamente mis preguntas con paginación y filtros.

1. Diseñar en el microservicio de Revisión la consulta de preguntas asignadas a un revisor, con paginación y filtros por estado, competencia y dificultad.
2. Implementar el endpoint GET /revisiones/asignadas que obtenga el revisor desde el token y consulte los datos de la pregunta al microservicio de Preguntas por API, sin acceder a su base de datos.
3. Documentar el endpoint en Swagger, con sus parámetros de paginación y filtros.
4. Implementar en el frontend (separado del backend) el listado del revisor, con tabla, filtros, paginador y estados con colores, consumiendo la API REST a través del gateway.
5. Crear pruebas unitarias automatizadas del servicio de consulta, de los filtros y de la paginación.

SSCRUM-63 Dado que reviso una pregunta asignada, cuando registro mi evaluación y mis observaciones, el sistema las guarda y quedan visibles para el autor.

1. Diseñar la estructura de persistencia del microservicio de Revisión para las entidades EvaluaciónRevisor (criterio, calificación, comentario) y Observación (texto, fecha).
2. Crear las clases de dominio EvaluaciónRevisor y Observación con sus atributos y relaciones con la revisión.
3. Implementar en la capa de servicios del microservicio la lógica para registrar la evaluación y las observaciones, y el endpoint POST /revisiones/{id}/evaluacion.
4. Implementar en el frontend la vista de lectura de la pregunta y el formulario de evaluación y observaciones.
5. Crear pruebas unitarias automatizadas de las clases de dominio y del servicio de evaluación.

SSCRUM-64 Dado que apruebo o rechazo una pregunta con mi evaluación, cuando confirmo la decisión, el sistema cambia su estado a “Aprobada” o “Rechazada” y notifica a los implicados.

1. Implementar el endpoint PUT /revisiones/{id}/decision que registre la decisión del revisor.
2. Solicitar al microservicio de Ciclo de vida el cambio de estado de la pregunta, validando que la transición desde “Pendiente de revisión” sea permitida (patrón State).
3. Publicar en el broker de mensajes el evento PreguntaAprobada o PreguntaRechazada, de modo que el microservicio de Notificaciones envíe el email al autor y al administrador y el de Auditoría registre el cambio.
4. Aplicar el patrón Observer para que los servicios suscritos reaccionen al evento sin acoplarse al microservicio de Revisión.
5. Implementar en el frontend los botones Aprobar y Rechazar con confirmación y mensaje de éxito.
6. Crear pruebas unitarias automatizadas de la decisión, la transición de estado y la publicación del evento (con el broker simulado).

SSCRUM-65 Dado que intento rechazar una pregunta sin escribir observaciones, cuando confirmo, el sistema no me lo permite y me exige registrar al menos una observación.

1. Implementar en la capa de servicios la regla de que un rechazo requiere al menos una observación no vacía.
2. Devolver un error de validación (400) con un mensaje claro, y documentarlo en Swagger.
3. Mostrar el mensaje de error en el formulario del frontend, indicando que la observación es obligatoria.
4. Crear pruebas unitarias automatizadas de la regla, con y sin observaciones.

SSCRUM-66 Dado que intento revisar una pregunta que no me fue asignada o que ya fue decidida, cuando envío la solicitud, el sistema la rechaza y no modifica la pregunta.

1. Implementar en el microservicio de Revisión la validación de que la pregunta esté asignada al revisor autenticado y siga pendiente de decisión.
2. Devolver los códigos de error 403 o 409 según el caso, con un mensaje claro, y documentarlos en Swagger.
3. Hacer que el gateway valide el token y el rol “Revisor” antes de enrutar la petición al servicio.
4. Crear pruebas unitarias automatizadas de las validaciones de asignación y de estado.

Nota: los títulos terminados en «...» están abreviados; búscalos por su inicio. En la HU 3 no se veía el número SSCRUM de cada subtarea.